UNIT 4

Constraint Satisfaction Problems & Game Theory

Prepared by
Mythili R

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

AGENDA

- *Local search for constraint satisfaction problems.*
- *Adversarial search*
  - ***Games***
  - ***Optimal decisions & strategies in games***
  - ***The minimax search procedure***
  - ***Alpha-beta pruning***
  - ***Additional refinements***
  - ***Iterative deepening***

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Adversarial search

- Adversarial search is a type of problem-solving approach used in **multi-agent environments** where two or more agents compete with each other to achieve **conflicting goals**.
- Unlike single-agent search problems, where an agent interacts only with the environment, adversarial search considers **the actions of other agents** that can affect the outcome.
- The objective is to find the **best possible move** assuming that the opponent is also playing optimally.
- Adversarial search is widely used in **game-playing AI systems** such as **Chess, Checkers, Tic-Tac-Toe, Go,** and other strategic decision-making applications.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Adversarial search

Characteristics of Adversarial Environments

- **Multi-agent:** at least two players act in the same environment.
- **Competitive:** the agents' goals conflict — one's gain is another's loss.
- **Sequential:** players take turns.
- **Deterministic:** outcomes of actions are predictable.
- **Perfect information:** all players see the complete game state.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Formal Definition of a Game

A game can be formally defined as a search problem with the following components.

Element	Description
S_0	Initial state – specifies how the game starts
PLAYER(s)	Indicates which player has the next move (MAX or MIN)
ACTIONS(s)	Returns the set of legal actions (moves) in state *s*
RESULT(s, a)	Defines the state that results from taking action *a*
TERMINAL-TEST(s)	True if the game is over (terminal state)
UTILITY(s, p)	Numerical value (payoff) for player *p* in terminal state *s* In chess, the outcome is a win, loss, or draw, with values +1, 0, or 1

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Game Tree

- The **game tree** is a representation of all possible game states and moves.
- **Nodes** represent game states, and **edges** represent possible actions.
- The **root** represents the initial state.
- **Leaf nodes** (terminal states) represent outcomes such as win, lose, or draw.
- Example:
  - In **Tic-Tac-Toe**, the initial state has 9 possible moves for MAX.
  - The game tree has fewer than 9! = 362,880 terminal nodes.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Agents in Adversarial Search

In two-player games:

- **MAX**: The agent that tries to maximize the utility (winning player).
- **MIN**: The opponent that tries to minimize MAX's utility (losing player).

- The **goal of MAX** is to choose an action that leads to the **highest possible utility**, assuming that MIN always tries to minimize it.
- Hence, adversarial search seeks to find an **optimal strategy** that performs best against an **optimal opponent**.

Zero-Sum Games

- A zero-sum game is one in which the total payoff to all players remains constant.
- A gain for one player means an equal loss for the other.
- Example: In Chess, if one player wins (+1), the other loses (–1).
  The total sum of payoffs = 0.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

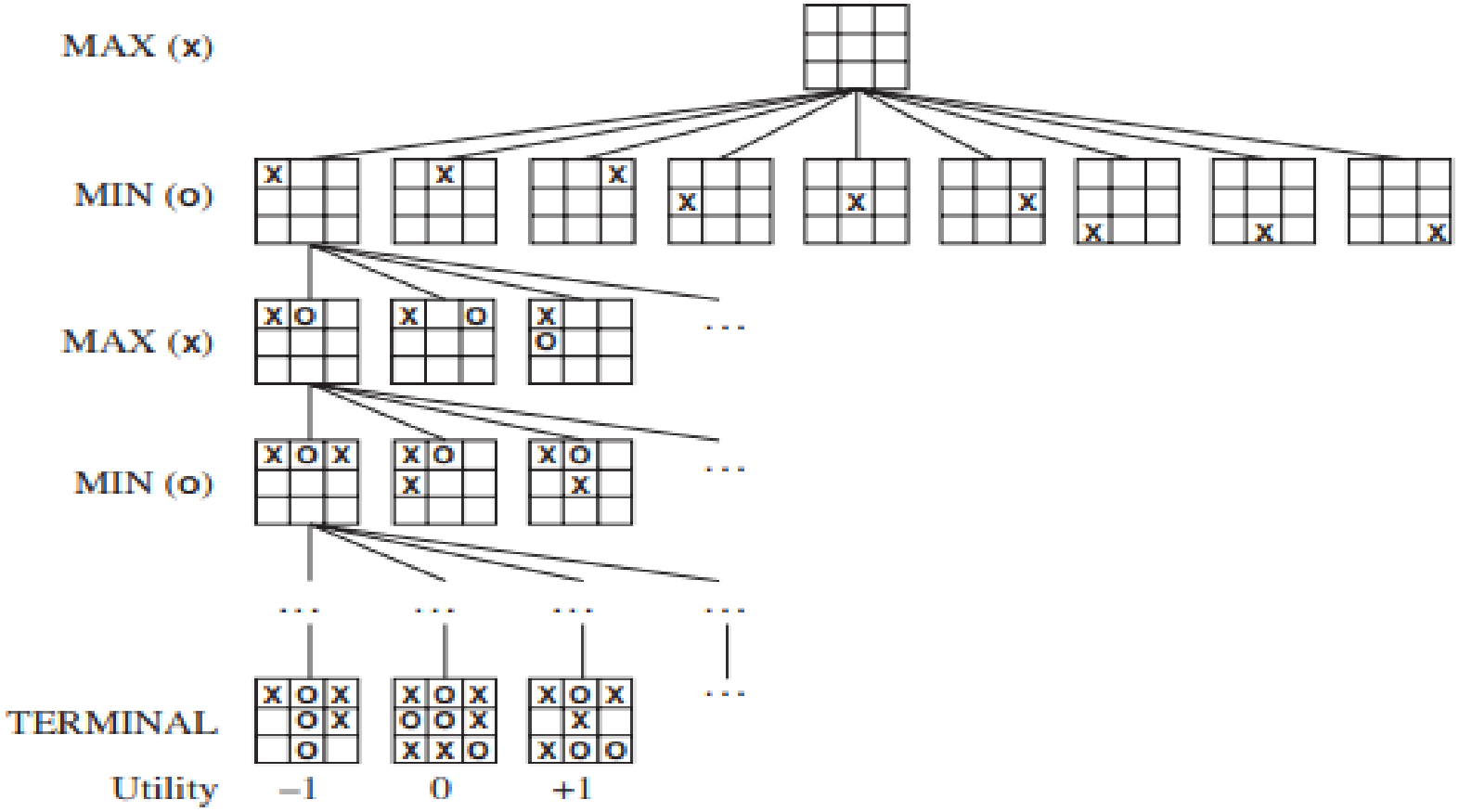


Figure 5.1 A (partial) game tree for the game of tic-tac-toe. The top node is the initial state, and MAX moves first, placing an X in an empty square. We show part of the tree, giving alternating moves by MIN (O) and MAX (X), until we eventually reach terminal states, which can be assigned utilities according to the rules of the game.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Optimal Decisions in Games (Minimax Algorithm)

- In adversarial search, every move made by one player affects the possible moves and outcomes available to the opponent.
- Thus, a rational player must choose actions that **maximize their advantage** while anticipating that the opponent will act to **minimize it**.
- This leads to the concept of the **Minimax Algorithm**, which provides an optimal decision rule for two-player, zero-sum games.

Goal of Minimax

- To find the **optimal strategy** for the MAX player, assuming the MIN player also plays optimally.
- The algorithm helps in **deciding the best move** by exploring possible future states of the game.
- It guarantees that even in the worst-case scenario (perfect opponent), the MAX player achieves the **best possible outcome**.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

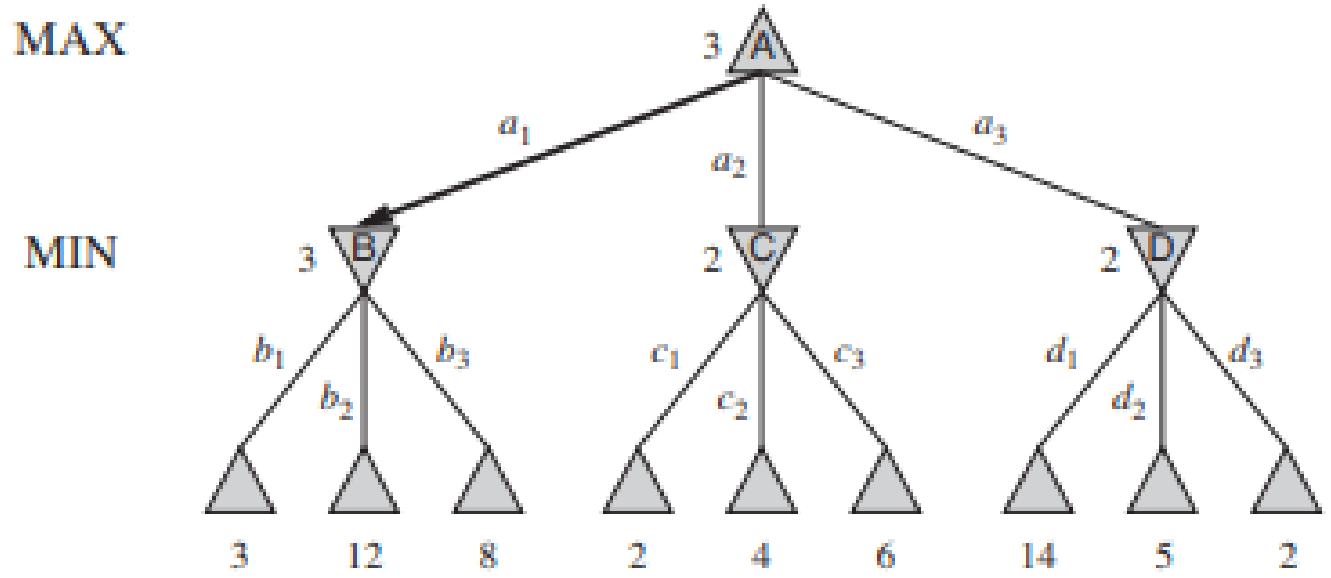


Figure 5.2 A two-ply game tree. The △ nodes are "MAX nodes," in which it is MAX's turn to move, and the ▽ nodes are "MIN nodes." The terminal nodes show the utility values for MAX; the other nodes are labeled with their minimax values. MAX's best move at the root is a_1, because it leads to the state with the highest minimax value, and MIN's best reply is b_1, because it leads to the state with the lowest minimax value.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Algorithm Steps

- **Generate** the entire game tree up to terminal states.
- **Apply** the utility function to evaluate terminal nodes.
- **Propagate** (back up) these values through the tree:
  - At MAX nodes → take the **maximum** child value.
  - At MIN nodes → take the **minimum** child value.
- **Select** the action at the root that gives the **highest minimax value**.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

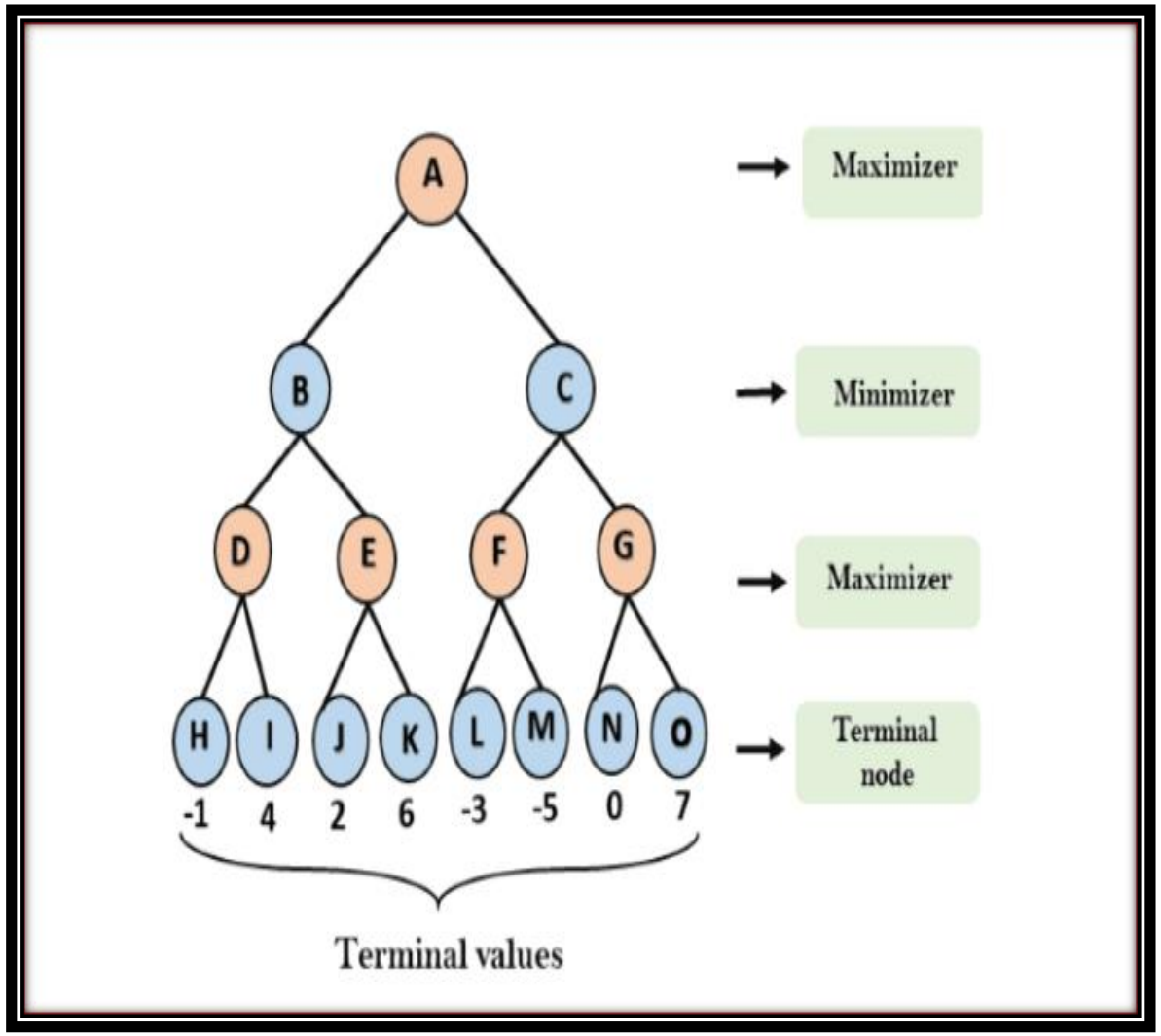
Minimax Principle

- The **Minimax algorithm** is a fundamental concept in artificial intelligence, particularly in game theory and decision-making processes.
- It's commonly used in two-player, zero-sum games, where one player's gain is the other player's loss (e.g., chess, checkers, tic-tac-toe).
- At its core, the **Minimax algorithm** is a way for an AI agent to make decisions by considering the possible outcomes of its own actions and its opponent's counter-actions, ultimately aiming to maximize its own advantage while minimizing the potential gain of the opponent.
- The name "**Minimax**" comes from the strategy of minimizing the potential loss (for the opponent) while maximizing the potential gain (for the AI player).
- It operates on the principle of adversarial search, where the AI agent simulates the opponent's moves and chooses its own actions accordingly.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

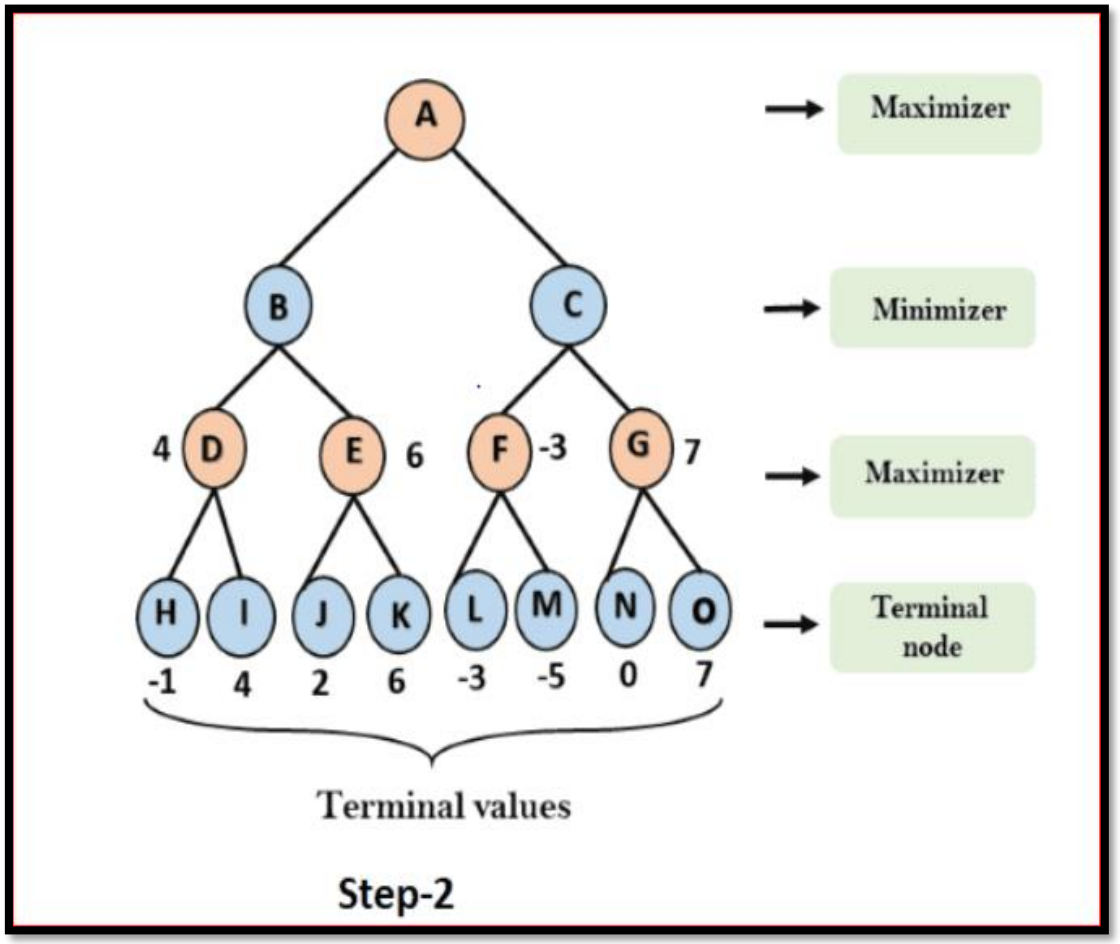
- **Step 1:** In the first step, the algorithm generates the entire game tree and applies the **utility function** to obtain the **utility values** for the terminal states.

- In the below tree diagram, let's designate **A** as the initial state of the tree. Suppose the **maximizer** takes the first turn, which initializes with a worst-case initial value of **-∞ or value =-** ∞, and the **minimizer** will take the next turn, initialized with a worst-case initial value of +∞ **or value** = +∞.



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

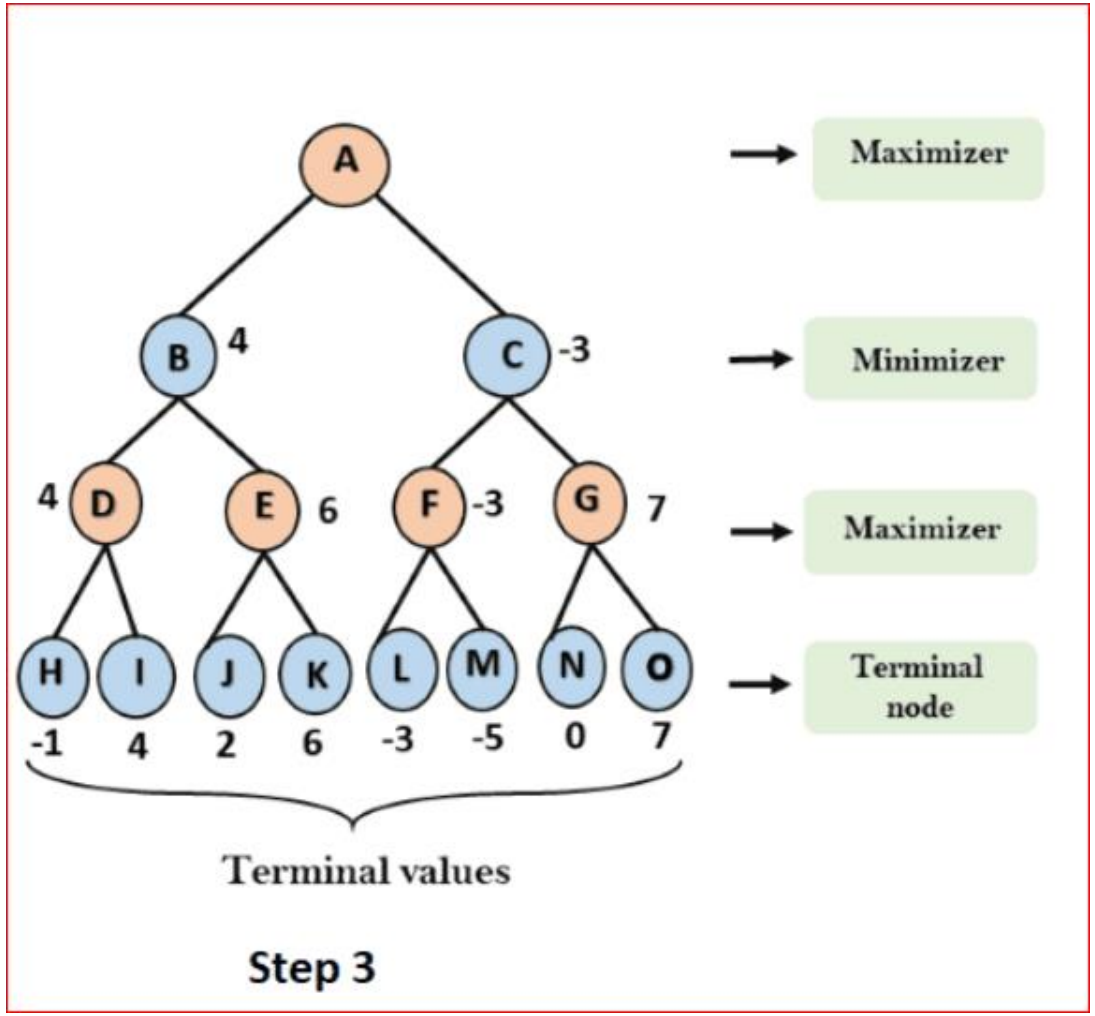
- **Step 2:** Now, we begin by determining the utility values for the **Maximizer**.
- With an initial value of -∞, we compare each value in the terminal states with the Maximizer's initial value, selecting the highest value encountered.
- This process ensures that the Maximizer evaluates the tree nodes to find the maximum utility among them.
- **For node D max(-1,- -∞) => max(-1,4)= 4**
- **For Node E max(2, -∞) => max(2, 6)= 6**
- **For Node F max(-3, -∞) => max(-3,-5) = -3**
- **For node G max(0, -∞) = max(0, 7) = 7**



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- **Step 3:** In the next step, it's the turn for the **Minimizer**. It will compare all node values with +∞ and find the values of the nodes at the **third layer**.
- **For node B = min(4,6) = 4**
- **For node C = min (-3, 7) = -3**

In the next step, algorithm traverse the next successor of Node B which is node E, and the values of α = -∞, and β = 3 will also be passed.

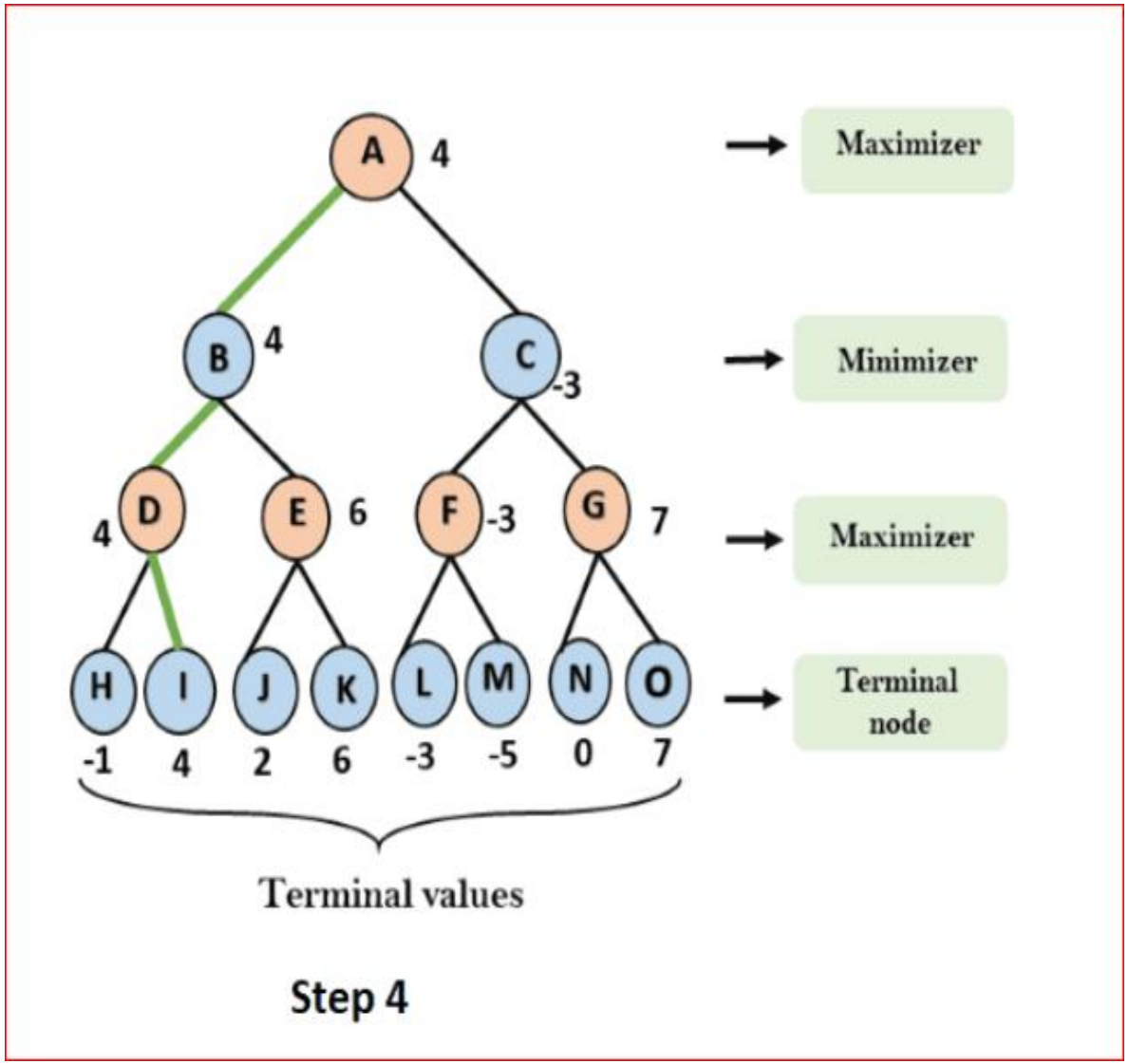


Step 3

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- **Step 4:** Now it's the turn for the **Maximizer**, and it will once again select the **maximum** of all node values to find the maximum value for the **root node**. In this game tree, there are only **4 layers**, hence we reach immediately to the root node, but in real games, there will be more than 4 layers.

- **For node A max(4, -3)= 4**



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Properties of Mini-Max algorithm:

- **Completeness**:
  - The Minimax algorithm is **complete**. It guarantees to find a solution (if one exists) within the finite search tree.
  - This means that it will explore all possible paths in the game tree until it reaches a terminal state.
- **Optimality**:
  - The Minimax algorithm is **optimal** when both opponents are playing optimally.
  - This means that it will select moves that maximize the AI player's chances of winning while minimizing the opponent's chances, assuming both players make the best possible moves.
- **Time Complexity**:
  - The time complexity of the Minimax algorithm is determined by the depth-first search (DFS) it performs on the game tree.
  - It is represented as $\mathbf{O(b^m)}$, where **b** is the **branching factor** of the game tree (i.e., the average number of possible moves per state), and **m** is the **maximum depth** of the tree (i.e., the length of the longest path from the root to a leaf).

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- **Space Complexity**:
  - The space complexity of the Minimax algorithm is also similar to DFS, given by **O(bm)**. It represents the amount of memory required to store the nodes and states explored during the search process.
  - As the algorithm explores deeper into the tree, more memory is required to store the nodes at each level.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

```
def minimax(node, depth, isMaximizingPlayer):
    if depth == 0 or is_terminal(node):
        return evaluate(node)
        if isMaximizingPlayer:
        bestValue = -infinity
        for child in node.children:
            val = minimax(child, depth-1, False)
            bestValue = max(bestValue, val)
        return bestValue
    else:
        bestValue = infinity
        for child in node.children:
            val = minimax(child, depth-1, True)
            bestValue = min(bestValue, val)
        return bestValue
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Alpha Beta Pruning

- Alpha Beta Pruning is a search optimization technique that improves the performance of the minimax algorithm.
- The minimax algorithm is a decision-making process commonly used in two-player, zero-sum games like chess.
- In such games, one player aims to maximize their score while the other seeks to minimize it.
- The minimax algorithm operates by recursively exploring all possible game states (represented as a tree structure) and assigning values to the leaf nodes based on the potential outcomes of the game.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- The algorithm then propagates these values up the tree to find the optimal move.
- However, as the complexity of the game increases, the number of possible states grows exponentially, leading to very high computational costs.
- This is where Alpha Beta Pruning becomes crucial.
- It reduces the number of nodes the minimax algorithm needs to evaluate by “pruning” branches that cannot influence the final decision.
- By eliminating unnecessary computations it simplifies the decision-making process, enabling faster and more efficient evaluations.
- As a result, Alpha Beta Pruning is practical for real-time applications, such as game-playing AI, where speed and efficiency are critical.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Alpha–Beta Pruning

Alpha (α)

- Represents the **best (highest)** value that the **Maximizing player (AI)** can guarantee **so far**.
- **Initial value:** $\alpha = -\infty$
- **Updated by:** Maximizer
- **Goal:** Increase α as better moves are found.

Beta (β)

- Represents the **best (lowest)** value that the **Minimizing player (opponent)** can guarantee **so far**.
- **Initial value:** $\beta = +\infty$
- **Updated by:** Minimizer
- **Goal:** Decrease β as better (smaller) moves are found.

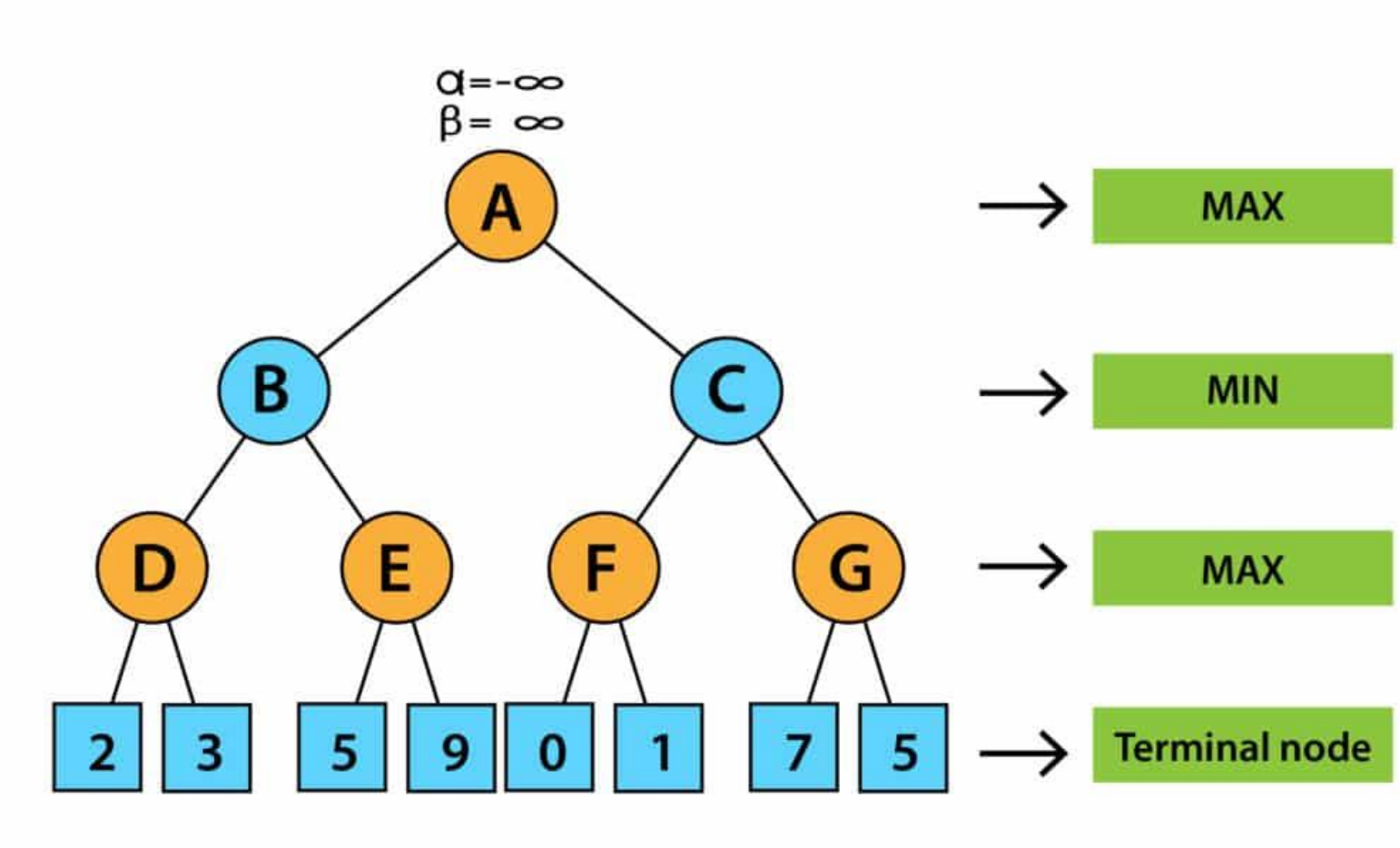
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

The Pruning Process

- As the AI explores the game tree, it keeps track of the current **α** and **β** values.
- At each node:
  - **Maximizer** updates $\alpha = \max(\alpha, \text{value})$
  - **Minimizer** updates $\beta = \min(\beta, \text{value})$
- **Pruning condition:**
  If $\boldsymbol{\alpha \geq \beta}$, further exploration of that branch is **stopped (pruned)**.
  This is because:
  - The **Maximizer** already has a better choice elsewhere (α is too high), or
  - The **Minimizer** will **never** allow the game to reach this state (β is too low).

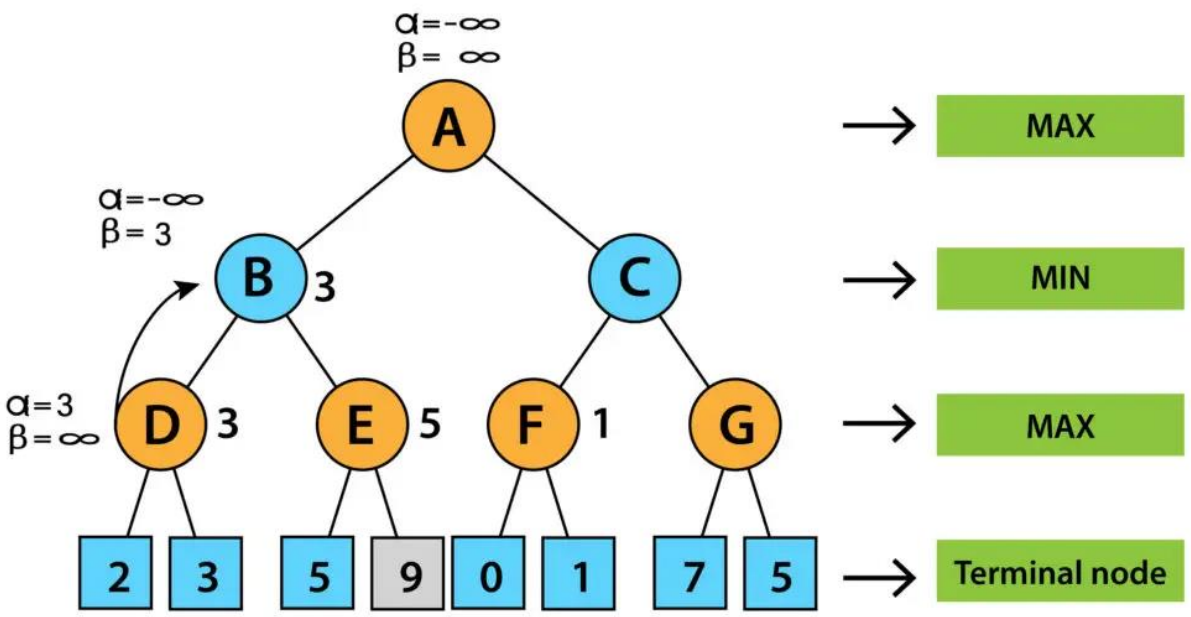
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- The first move initially, we will define the worst case α = −∞ and β = +∞. If alpha is greater than or equal to beta, we will prune the node.



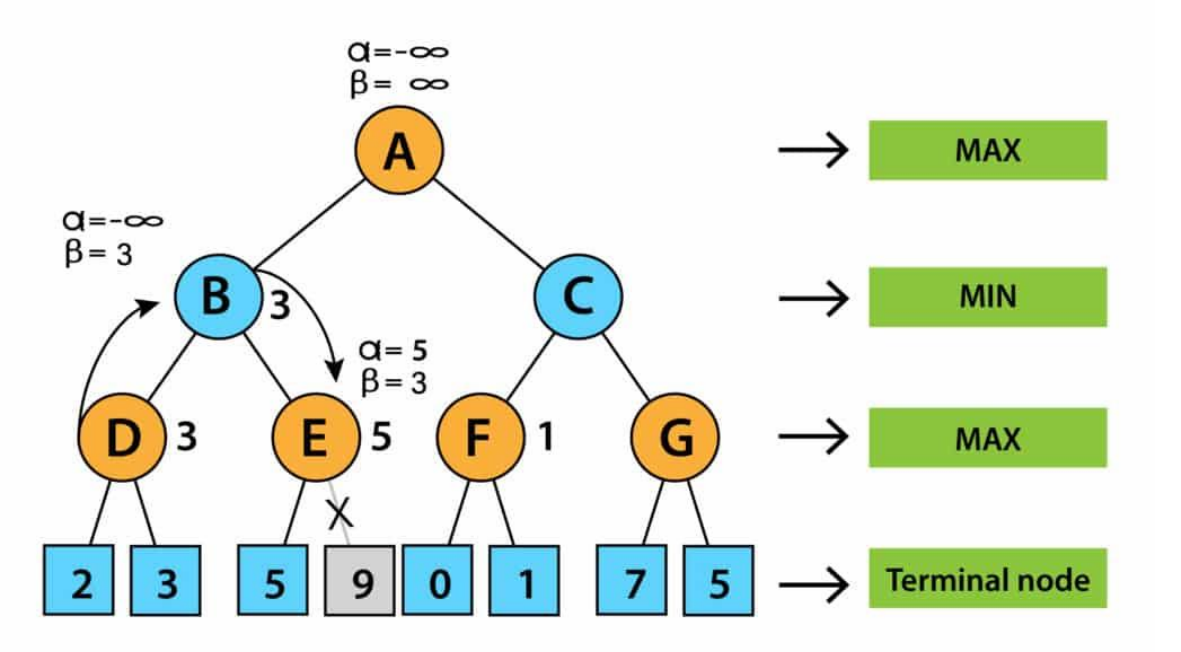
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- We didn't prune it since the initial value of alpha is less than beta. Now it's turn for MAX. Therefore, we will calculate the value of alpha at node D. At node D, the value of alpha will be max (2, 3) = 3.
- Now, node B's turn is MIN. That means that the value of alpha beta at node B will be min (3, ∞). Therefore, alpha will be – ∞ and beta will be 3 at node B.
- Next, algorithms will pass the values of α= -∞ and β= 3 to the next successor of Node B, that is node E.



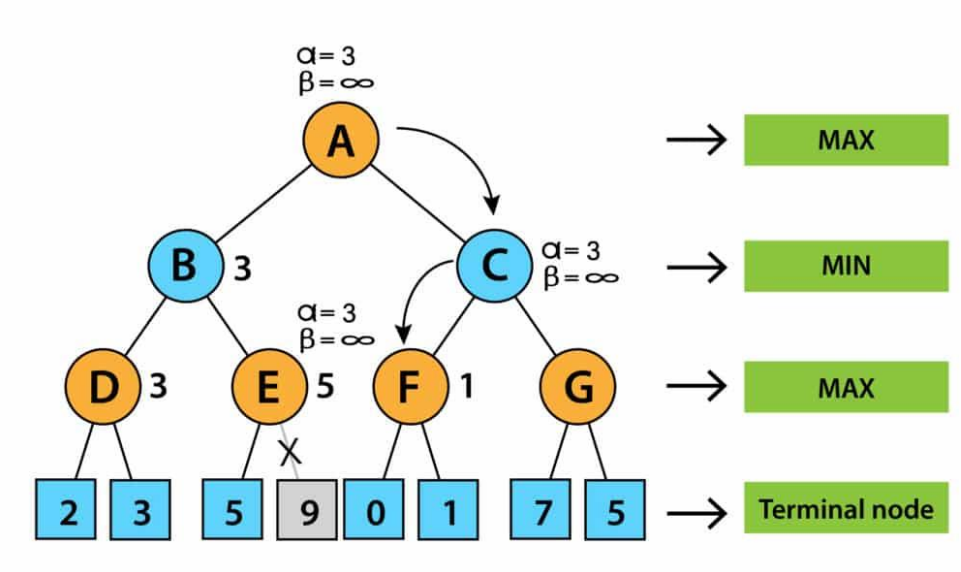
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- Now it’s turn for MAX. Therefore, we will search for MAX at node E. The value of alpha at E is – ∞ and will be compared with 5. So, MAX (- ∞, 5) will be 5. Thus, at node E, alpha = 5, Beta = 5.
- We now know that alpha is greater than beta and is also a pruning condition, so we can prune the right successor of node E, and the algorithm will not be visited, and the value of node E will be 5.



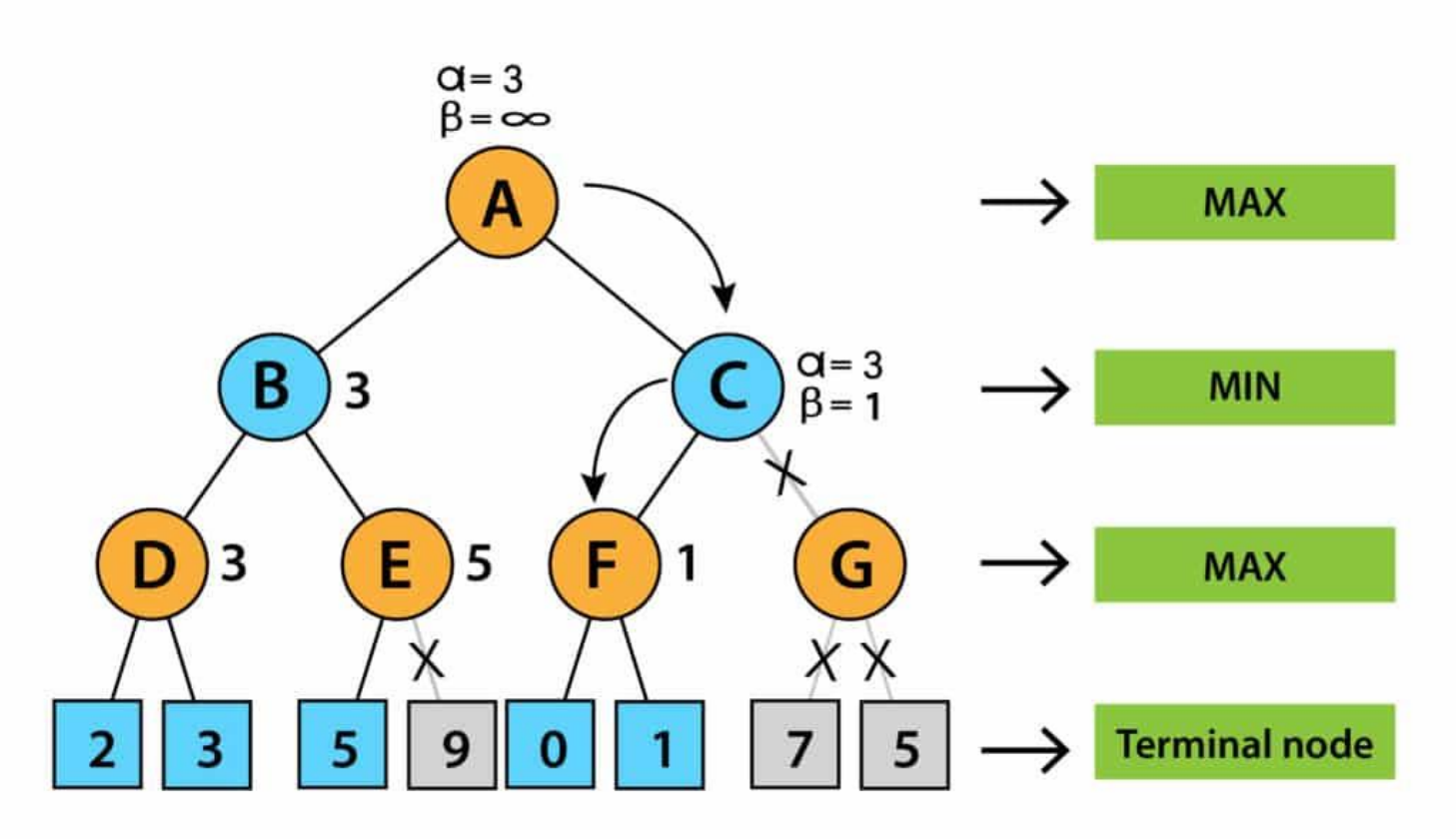
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- In the next step, the algorithm again comes from node B to node A. The alpha of node A is changed to the max value of MAX(- ∞, 3). Now, alpha and beta at node A are (3, + ∞) and will be transmitted to node C. The same values will be transferred to node F.
- The value of alpha will be compared to the left branch, which is 0 at node F. Now, MAX (0, 3) will be 3 and is compared with the right child (1) and MAX(3, 1) = 3 still α is 3, but the node value of F will be 1.



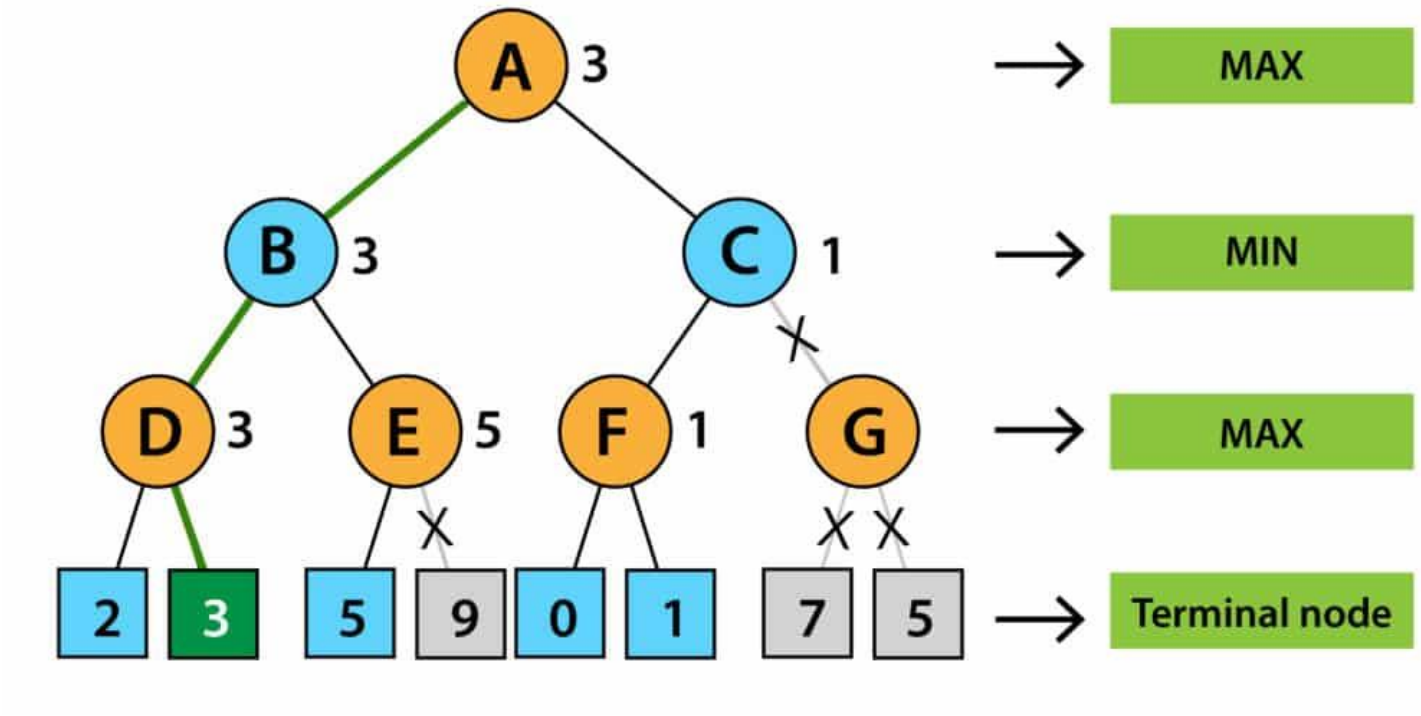
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- Afterward, node F will pass the node value 1 to C and compare it to the beta value at C. Now it's turn for MIN. So, MIN (+ ∞, 1) will be 1. Now, at node C, α = 3, β= 1, and alpha is greater than beta, which again satisfies the pruning condition. Then, the successor of node C, G, will be pruned, and the algorithm did not calculate the whole subtree G.



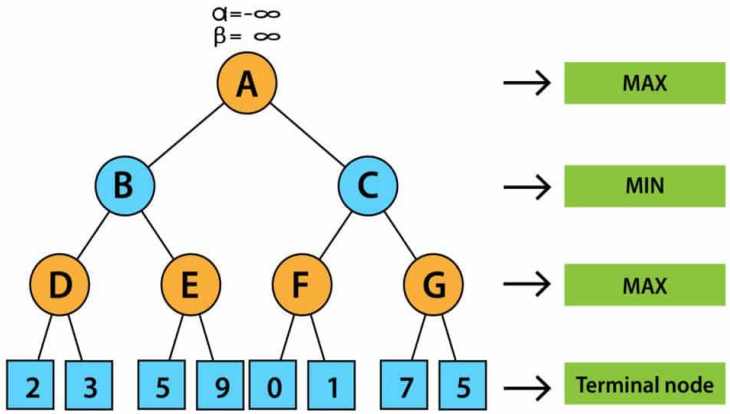
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

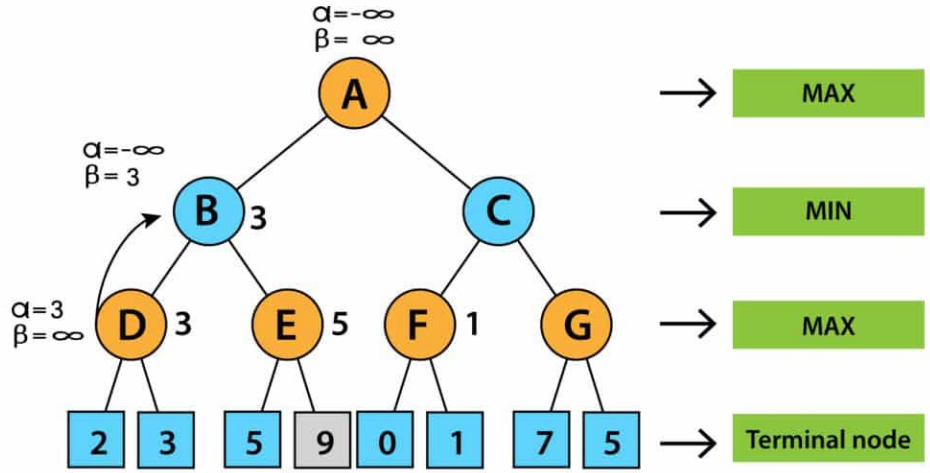
- Now, C will return its node value to A, and A will calculate MAX(1, 3), resulting in 3.

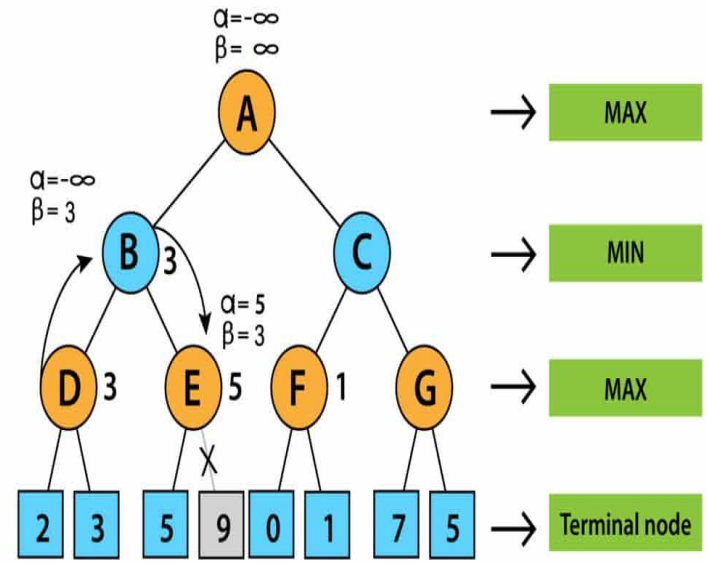


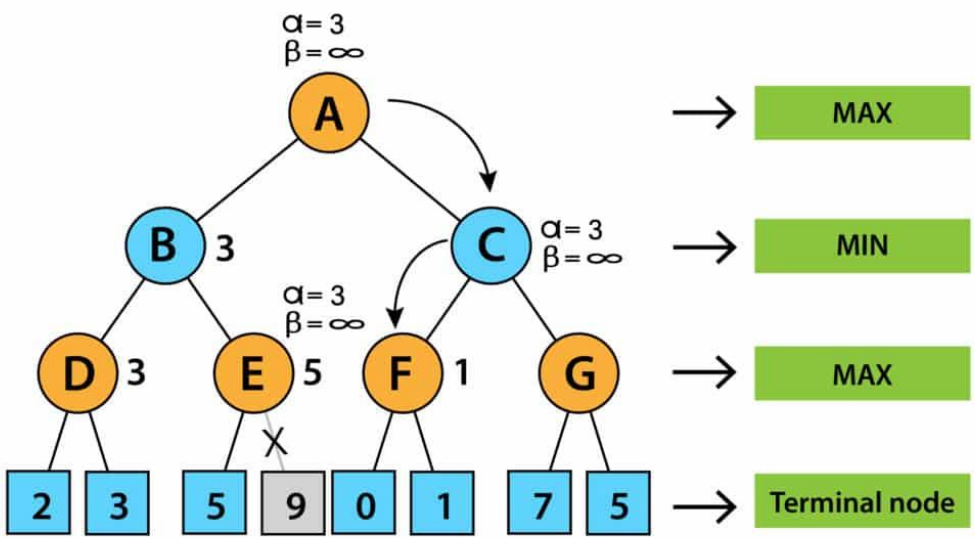
This above-represented tree is the final tree that has nodes that are computed and nodes that are not computed. Therefore, in this example, the optimal value of the maximizer would be 3.

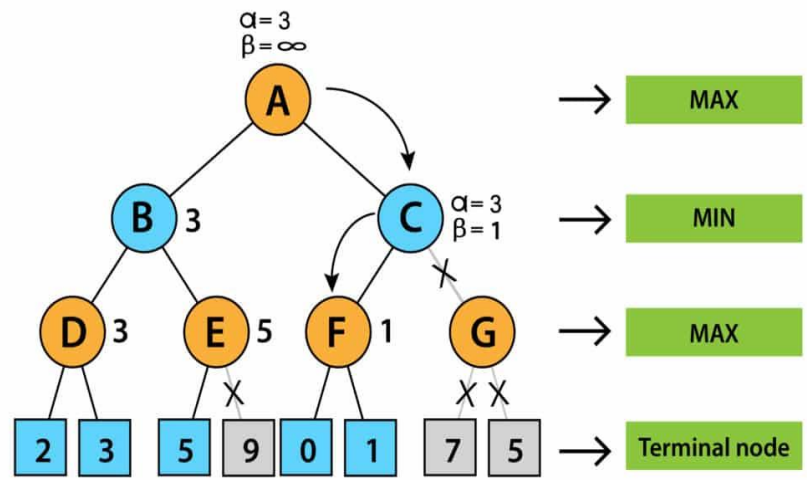
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

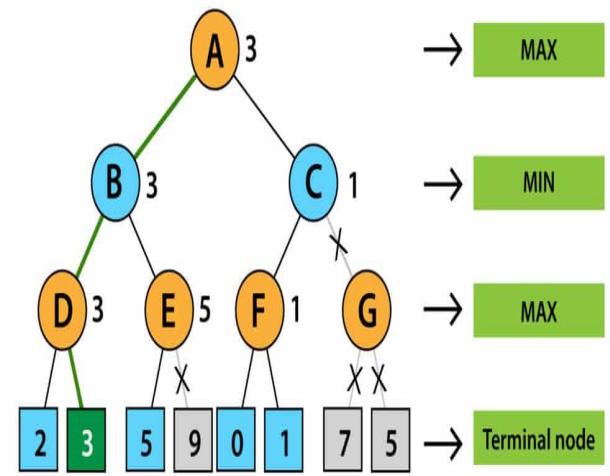












This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.